

API DE JAVA - TEMA 5

STREAM Y EXPRESIONES LAMBDA

ÍNDICE

1. stream 1 operaciones intermedias
2. stream 2 operaciones terminales
3. expresiones lambdas explicación general y predicador (yo)
4. Expresiones lambda 2 (interfaces funcionales)

EXPRESIONES LAMBDA

Regla de Oro ⇒ Una expresión lambda es como una clase con un único método público escrito de forma ultra compacta.

CARACTERÍSTICAS CLAVES

CARACTERÍSTICAS	DESCRIPCIÓN
Recibe argumentos	Acepta cero, uno o más parámetros.
Devuelve valor	Puede tanto devolver un valor como no devolver nada
Acceso a variables locales	Esto nos indica que permite leer variables donde se declara , SIN EMBARGO no modificarlas (<i>deben ser final o effectively final</i>)
Paquete principal	java.util.function

¿Qué significa **final o effectively final**? ⇒ A esto nos referimos a que buscamos que una vez asignada la función lambda **no cambie su valor**. Por ello agregar modificador final o seguir esta metodología.

Comprender: entendemos a estas como funciones anónimas que pueden **tratarse** como valores y **pasarse argumentos** a los métodos.

En java consideramos a las **funciones lambda** como **interfaces funcionales**. (Esto quiere decir que son interfaces con **un único método abstracto SAM**).

Comprender: la presencia de un **solo método abstracto** garantiza que la interfaz SI pueda utilizar expresiones lambda.

Comprender Finalmente → Una expresión lambda es como una clase con un único método público escrito de forma ultra compacta.

LO QUE NOS PROPORCIONA LA API DE JAVA

⇒ Java nos proporciona un conjunto de interfaces funcionales integradas en el paquete “`java.util.function`”, **estas son interfaces con un unico metodo que se debe implementar**, este incluye:

- **Supplier <T>** ⇒ utilizada para generar objetos sin requerir de argumentos
- **Consumer <T>** ⇒ opuesto a *supplier*, acepta argumentos tipo T sin devolver ningún valor como retorno.
- **Function <T,R>** ⇒ permite definir una función que acepta un parámetro de tipo T y devuelve un resultado tipo R (*se le pueden aplicar transformaciones u operaciones*)
- **Predicate <T>** ⇒ estos devuelven **obligatoriamente un booleano** le damos un objeto tipo T (*utilizado para filtrar elementos en una colección*).
- **BiFunction <T,R,S>** ⇒ nos permite definir una función que acepte como parámetro T y R, y nos devuelve un resultado tipo S. (*utilizado de normal para operaciones de agregación u operadores binarios como sumas, restas, etc*)

Respecto a los términos T,R,S: son **marcadores de posición**, para tipos de datos reales. Huecos para rellenar cuando utilizamos la interfaz. Básicamente son **CONVENCIONES**.

- T ⇒ Type ⇒ Tipo principal de entrada
- R ⇒ Return ⇒ El tipo de retorno o resultado
- S ⇒ Second ⇒ Segundo tipo de entrada

CLASES ANÓNIMAS – STEP PARA COMPRENDER LAMBDA

⇒ Son aquellas clases que se crean en el momento **SIN NOMBRES**. A esto se le llama clase anónima.

Entendiendo una clase anónima:

De normal hacemos lo siguiente:

{ } Java

```
// Una interfaz con UN solo método
public interface Saludador {
    String saludar(String nombre);
}
```

} IM

{ } Java

```
// Una clase que la implementa
public class SaludadorAmable implements Saludador {
    @Override
    public String saludar(String nombre) {
        return "¡Hola, " + nombre + "!";
    }
}
```

} imp + im

{ } Java

```
// La usas así
Saludador miSaludador = new SaludadorAmable();
String resultado = miSaludador.saludar("Ana");
System.out.println(resultado); // ¡Hola, Ana!
```

Las clases anónimas quedaría tal que:

{ } Java

```
Saludador miSaludador = new Saludador() {
    @Override
    public String saludar(String nombre) {
        return "¡Hola, " + nombre + "!";
    }
};

String resultado = miSaludador.saludar("Ana");
System.out.println(resultado); // ¡Hola, Ana!
```

Entender ⇒ Lo que hemos hecho es que en vez de crear una clase *SaludadorAmable* la creamos **en el momento**, sin nombre.

COMPRENDIENDO LA SINTAXIS DE LAMBDA EN JAVA

⇒ Pues bien, siguiendo el ejemplo anterior vamos a analizar y entender más a detalle cómo se forma un **lambda**:

Lo primero que realizaremos es entender que en nuestro código en muchas de las ocasiones **sobran cosas que java ya sabe y no necesita**.

GRABAR A FUEGO: para poder llevar a cabo una **expresión lambda** la interfaz solo podrá llevar en sí **UN MÉTODO PÚBLICO**

Proceso de descomposición de lambda:

```
{ } Java

Saludador miSaludador = new Saludador() {
    @Override
    public String saludar(String nombre) {
        return "¡Hola, " + nombre + "!";
    }
};

// Java ya sabe que es Saludador
// Sobra: solo hay un método
// Java ya sabe el nombre del método
// ← ESTO es lo único importante
```

Explicación:

1. si analizamos el siguiente código **java** ya comprende que es **Saludador** ya que se menciona al inicio por lo que la siguiente es irrelevante. (**PRIMERA COSA A DESCARTAR**)
2. Si volvemos a analizar como sabemos << para poder utilizar expresiones lambdas deben estar en interfaces de un **único método** >> por lo que ya realmente se sabe el método que tenemos en la clase. (**SEGUNDA COSA A RECORTAR**).

Entendiendo esto podemos empezar ya a construir nuestra primera versión de lambda (**versión larga**).

```
{ } Java

//      lo que recibe  →  lo que hace
//
Saludador miSaludador = (String nombre) -> {
    return "¡Hola, " + nombre + "!";
};
```

Explicación: aquí analizamos cómo utilizamos la **clase de la interfaz + variable nombre**; luego encontramos los **parámetros del método** finalmente agregamos la flecha donde **empezará la función del método**.

Básicamente es lo mismo pero sin relleno.

Otra forma que nos permitirá recortar sería mediante el **tipado de los parametros**: e

```
// Antes (Paso 3):
Saludador miSaludador = (String nombre) -> {
    return "¡Hola, " + nombre + "!";
};

// Ahora (sin el tipo):
Saludador miSaludador = (nombre) -> {
    return "¡Hola, " + nombre + "!";
};
```

Explicación: básicamente si pensamos en el **flujo** entendemos que java realmente sabe el tipo dado que al crear la interfaz le indicamos el tipo que recibe. Por lo que podremos quitarlo.

PRACTICANDO LAMBDA Y ACONSEJANDO SOBRE ELLOS

Categoría: Iniciación

PRACTICA 1: Lambda con retorno

```
@FunctionalInterface
interface Calculadora {
    int calcular (int a, int b);
}
```

creamos la interfaz

```
Calculadora suma = (a,b ) -> { // Me refiero a a y b
    return (a + b);
};

System.out.println(suma.calcular(5, 10));
```

Es importante reconocer el punto y coma tras la llave.

⇒ Este primer lambda lo vemos de forma muy sencilla, como podemos observar no hace falta declarar en sus parámetros los tipos de las variables ya que lo hicimos con anterioridad.

Importante sobre ello:

⇒ Durante el testeo del ejercicio tuve el error de pensar que los valores había que instanciarse (el a y b). Error ya que hay que pensar que esto es una interfaz y que los valores se utilizan al hacer la llamada a su método.

⇒ Otro error que podemos encontrar es que si usamos las variables que están inicializadas en el lambda se pueden pisar y dar un error de **scope**.

PRACTICA 2: Lambda sin retorno

```
@FunctionalInterface
interface Accion {
    void ejecutar (String mensaje);
}
```

```
/*Este lambda esta enfocado en uno sin retorno*/
Accion gritar = (valorA) -> {
    System.out.println(valorA);
};
```

⇒ En este ejemplo tenemos un tipo de lambda en el cual no tenemos un valor como retorno.

PRACTICA 3: Lambda sin parámetros

```
@FunctionalInterface
interface Proveedor {
    String proveer();
}
```

```
Proveedor saludo = () -> {
    return "Hola";
};
```

Categoría: Media (utilizar las interfaces de Java)

⇒ En esta categoría nos vamos a enfocar en utilizar las interfaces ofrecidas por Java:

1. **Function**
2. **Predicate**
3. **Consumer**
4. **Supplier**

Para ello llevaremos a cabo la siguiente importación:

```
import java.util.function.*;
```

FUNCTION (encargado de transformar)

STREAMS DESDE CERO

⇒ Para comprender los **streams** tenemos que entender el problema que buscan solucionar: **eliminar código repetitivo**.

Visión de stream: para ver a un stream debemos comprenderlo como un **flujo de datos**.

Características:

1. Trabaja con lista de **elementos** donde van pasando de uno en uno
2. En cada operación se les hace algo
3. Se recoge en **un resultado**.

Como crear nuestros streams:

Comprender: cualquier lista puede convertirse en un Stream

¿Cómo convertir una lista a Stream?

⇒ Para ello aplicaremos el .stream().

Ejemplo:

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);  
  
// Crear el stream a partir de la lista  
Stream<Integer> flujo = numeros.stream();
```

⇒ Ahí lo que podemos observar es cómo hemos transformado la lista en un **stream**, pero como tal no está realizando nada especial.

Para poder trabajar con los streams encontramos diferentes métodos:

[Métodos](#)

*Ejemplo del bucle **forEach**:*

```
nombres.stream()           // 1. Pon los elementos en  
    .forEach()              // 2. Para cada elemento..  
        nombre -> System.out.println(nombre) // 3.  
    );
```

=====

=====

SCANNER Y MATCHERROR DE INPUT

=====